Ders 8 Genişletilmiş Notu: Bağlı Listeden Silme, Dairesel ve Çift Yönlü Bağlı Listeler

Ders 8 Genişletilmiş Notu: Bağlı Listeden Silme, Dairesel ve Çift Yönlü Bağlı Listeler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bağlı Liste İşlemleri Neden Bu Kadar Önemli?
- 2. Bağlı Listeden Silmenin Mantığı
- 3. Silme Algoritmasını Adım Adım Kurmak
- 4. Dairesel Tek Yönlü Bağlı Liste
- 5. Dairesel Listede Başa Ekleme
- 6. Dairesel Listede Sona Ekleme
- 7. Bellekteki Adreslerle İlgili Bir Soru
- 8. Dairesel Listeyi Yazdırmak
- 9. Çift Yönlü Bağlı Liste
- 10. Çift Yönlü Listede Başa Ekleme

1. Bağlı Liste İşlemleri Neden Bu Kadar Önemli?

Önceki derste bağlı listeden (linked list) eleman silmenin nasıl olacağını konuşmaya başlamıştık. Hoca bu derse o noktadan devam etti ve konuyu neden tamamlamak istediğini açıkladı. Bağlı liste mantığı yalnızca bu konuya ait değildir. Yığın, kuyruk, ağaç ve graf gibi veri yapılarını tasarlarken de aynı mantığı kullanırız.

> **Hocanın vurgusu:** Bağlı liste mantığını bilmemiz önemlidir. İleride yığın, kuyruk, ağaç ve graf anlatılırken ekleme ve silmenin nasıl yapıldığı tekrar tekrar anlatılmayacak.

2. Bağlı Listeden Silmenin Mantığı

2.1 Bir Örnek: 9'u Silmek

Hoca, önceki dersteki örnekle başladı. `head` 1000'i gösteriyor. 1000 numaralı adreste 7 var ve 1500'ü gösteriyor. 1500'de 9 var ve 1700'ü gösteriyor. 1700'de 12 var ve NULL'u gösteriyor, yani son eleman budur.

```
head -> [1000: 7] -> [1500: 9] -> [1700: 12] -> NULL
```

Silinecek değer 9 olsun. Önce 9'u bulmamız gerekir. Baştan bakarız: Bu 9 mu? Hayır. Sonraki 9 mu? Evet. Bulduktan sonra olması gereken şudur: 7 artık 9'u değil, 9'un gösterdiği 12'yi göstermek zorundadır.

Bunu yapabilmek için 9'u kimin gösterdiğini bilmemiz gerekir. Yani 9'a nereden geldiğimizi tutmalıyız. Bu örnekte 1000 numaralı adresteki elemandan geldik. O elemana gidip "Artık 1500'ü gösterme, çünkü 1500 silinecek. 1500'ün gösterdiği 1700'ü göster" deriz.

> **Hocanın vurgusu:** Silme işleminde bir önceki elemanın (previous) adresi saklanmalıdır. Bunu unutmayacağız. Aksi hâlde önceki eleman, silinen elemandan sonrakine bağlanamaz.

2.2 Silinen Yeri Serbest Bırakmak

Bağlantıyı düzeltmek yetmez. C dersinden hatırlayacağınız gibi, bellekte bir yer ayırdığınızda (allocate) heap bölgesinde, o yer müsaitse, istediğiniz kadar yer tutulur. İşiniz bitince orayı `free` ile serbest bırakmalısınız ki başka ihtiyaçlar için kullanılabilsin. "Başkaları" derken kastedilen, çalışma sırasındaki (runtime) başka yer ayırma işlemleridir.

Çalışma sırasında sürekli yer tutup hiç serbest bırakmazsanız, bir süre sonra bellek taşar ve yer istediğinizde size yer verilemez hâle gelir.

2.3 Önce Durumları Belirlemek

Hoca silme yaparken dikkat edilecekleri şöyle sıraladı:

- **Liste boş değilse, önceki elemanın adresi saklanmalı.** Önceki eleman, silinecek elemandan sonraki elemanı gösterecek hâle getirilir.
- **Liste boş olabilir.** Listenin her zaman dolu olacağına dair bir kural yoktur. Sile sile hiç eleman kalmamış olabilir. Bu yüzden en başta kontrol edilir: `head` NULL ise bir şey yapmadan durulur.
- **Eleman listede olmayabilir.** Örneğin 15 aranıyor olsun: 7 mi? Hayır. 9 mu? Hayır. 12 mi? Hayır. 12'nin gösterdiği adres NULL'dur, yani bakacak eleman kalmamıştır. Durma şartlarından biri bu olmalıdır.

Buradan genel bir tasarım ilkesi çıkar. Önce uç noktaları ve olası durumları belirlersiniz: Eleman yoksa ne olacak? Eleman varsa ne olacak? Liste boşsa ne olacak? Algoritmayı sonra, bu durumları karşılayacak şekilde kurarsınız.

> **Hocanın vurgusu:** Önce ana durumları belirleyin, sonra algoritmayı bu durumları karşılayacak şekilde tasarlayın. Böylece yama yapılmış bir algoritma ortaya çıkmaz.

3. Silme Algoritmasını Adım Adım Kurmak

Hoca algoritmayı C kodu mantığıyla yazılmış bir akış diyagramı üzerinden anlattı, çünkü durumların hepsi böyle daha rahat görülüyordu. Bunu bir `delete` fonksiyonuna çevirmeyi öğrencilere bıraktı. Kodu yazmanın `insert` fonksiyonunu yazmaktan farklı olmadığını da ekledi.

Hocanın zihinde tuttuğu model üç düğümdür: A, B ve C. Silinecek eleman B ise A artık C'yi gösterecektir. Burada A önceki eleman, yani `prev`'dir. B ise o anda baktığımız elemandır, yani `current`.

Fonksiyon önceden oluşturulmuş bir liste üzerinde çalışır. Silinecek değer `value` okunur ve yalnız bu değeri silmek için gereken işlemler yapılır.

3.1 Birinci Kontrol: Liste Boş mu?

İlk kontrol `head`'in NULL olup olmadığıdır. Liste boşsa ekrana "Empty List" yazılır ve başka işlem yapılmaz.

3.2 İkinci Kontrol: Silinecek Eleman İlk Eleman mı?

Liste boş değilse yeni bir problem ortaya çıkar. `head` A'yı gösteriyor olsun. A silinecekse `head` artık B'yi göstermelidir. Demek ki `head`'in gösterdiği değerin silinecek değer olup olmadığı da ayrıca kontrol edilmelidir. Değer eşitse sırayla şunlar yapılır:

```
tmp = head;          /* silinecek adres aklımızda kalsın */
head = head->next;   /* head artık ikinci elemanı gösterir */
free(tmp);           /* eski ilk eleman serbest bırakılır */
```

Hocanın sayısal örneği şöyleydi. `head` 1000'i gösteriyor, 1000'de 5 var ve 1200'ü gösteriyor. Silinecek değer 5.

- `head->value`, 1000 numaralı adresin değeridir, yani 5. $5 = 5$ olduğu için bu dala girilir.
- `tmp = head`: `tmp` 1000 olur. Bu adres silineceği için aklımızda tutmak zorundayız.
- `head = head->next`: 1000'in `next`'i 1200'dür, `head` 1200 olur.
- `free(tmp)`: 1000 serbest bırakılır.

Soru: `head`'in zaten kendi değeri yok mu? `head` tek bir gösterici (pointer) değil mi, içinde sadece adres yok mu? O hâlde `head->value` nasıl 5 oluyor?

Cevap: `head`'in içinde gerçekten yalnızca adres vardır. Hoca bunu ayrı bir çizimle gösterdi: `head` değişkeni kendisi örneğin 800 numaralı adreste duruyor olsun, bu bizi hiç ilgilendirmez. İçinde 1000 yazıyor, yani 1000'i gösteriyor. `head`'in `value`'su dediğimizde kastedilen, 1000 numaralı adresin `value`'sudur. O da 5'tir. `head->next` de aynı şekilde 1000 numaralı adresin `next`'idir, yani 1200.

3.3 Üçüncü Durum: Arada veya Sonda Silme

Silinecek eleman ilk eleman değilse, önce nereden geldiğimizi saklarız. A'ya zaten baktığımız için başlangıç değerleri şunlardır:

```
prev = head;
current = head->next;
```

Örnek liste: `head` 1000'i gösteriyor. 1000'de 5 var ve 1200'ü gösteriyor. 1200'de 7 var ve 1500'ü gösteriyor. 1500'de 9 var ve NULL'u gösteriyor.

Silinecek değer 7 ise `head->value` yani 5, 7'ye eşit değildir. 1000 ile işimiz bitti. `prev` 1000, `current` 1200 olur ve bakmaya 7'den başlarız.

Bundan sonra elemanı bulana kadar bir döngü kurmak gerekir. Döngünün iki koşulu vardır:

```
(current != NULL) && (current->value != value)
```

Birinci koşul listenin bitmediğini söyler, çünkü `current` NULL ise bakacak eleman kalmamıştır. İkinci koşul, aranan elemanın henüz bulunmadığını söyler.

> **Ders sonrası not:** Hoca bu iki koşula "durma şartı" dedi; doğrusu bunlar döngünün devam şartlarıdır. Döngü, `current` NULL olunca ya da `current->value` aranan değere eşit olunca durur.

Koşul doğruysa o elemanla işimiz bitmiştir. Onu `prev` hâline getirir, bir sonrakine geçeriz:

```
prev = current;
current = current->next;
```

Soru: Niçin önce `current`'ın NULL'dan farklı olması kontrol edildi? **Cevap:** Bir öğrenci doğru cevabı verdi: `current` NULL olsaydı `value`'suna erişilemezdi. Hoca da aynısını söyledi. `current` NULL ise `value` diye bir şey yoktur. Koşullar "ve" ile bağlandığı için önce NULL kontrolü yapılmalıdır.

> **Ek bilgi (derste anlatılmadı):** C dilinde `&&` işleci soldan sağa değerlendirilir. Sol taraf yanlışsa sağ taraf hiç hesaplanmaz. Buna kısa devre değerlendirmesi (short-circuit evaluation) denir. Bu yüzden `(current != NULL) && (current->value != value)` yazımında `current` NULL olduğunda `current->value` okunmaz. Koşulların sırası ters yazılırsa NULL gösterici üzerinden okuma yapılmış olur.

3.4 Bir İz: 1000, 1200, 1500 Listesinden 9'u Silmek

Hoca ilerlemeyi görmemiz için aynı listede silinecek değeri 9 aldı ve `prev`, `current`, `current->value` değerlerini tahtaya yazdı.

Adım	`prev`	`current`	`current->value`	Karşılaştırma
Başlangıç	1000	1200	7	$7 \neq 9$, ilerle

1. ilerleme	1200	1500	9	$9 = 9$, döngüden çık

Yukarı çıkıp koşula yeniden bakıldığında `current` NULL'dan farklıdır, çünkü 1500'dür. Fakat `value`'su aranan değerden farklı değildir. Bu yüzden döngüden çıkılır.

3.5 Döngüden Çıkınca Ne Yapılır?

Hoca burada öğrencilere sordu: Döngüden çıkınca ne yapmalıyız? Neden çıktığımızı bulmalıyız. Peki `current->value`'nun aranan değere eşit olup olmadığına mı, yoksa `current`'ın NULL olup olmadığına mı bakmalıyız?

Bir öğrenci doğru gerekçeyi verdi: `current` NULL olduğu için çıkmış olabiliriz, o durumda `value`'ya bakılamaz. Bu yüzden önce `current == NULL` kontrol edilir. Döngü koşulundaki mantığın aynısıdır.

> **Hocanın vurgusu:** Döngüden çıkınca da önce `current`'ın NULL olup olmadığına bakılır. `current` NULL ise `value`'su yoktur, karışıklık yaratmamak gerekir.

- **`current` NULL ise:** Eleman listede yoktur. Ekrana "not exist" gibi bir mesaj yazılır.
- **`current` NULL değilse:** Eleman bulunmuştur. Önceki düğüm, silinecek düğümün sonrasına bağlanır ve silinen düğüm serbest bırakılır.

```
prev->next = current->next;
free(current);
```

Örneğimizde `prev` 1200, `current` 1500'dür. 1500'ün `next`'i NULL olduğu için 1200'ün `next`'i NULL olur, yani 7 artık NULL'u gösterir. `free(current)` ile de 1500 numaralı adres kullanıma açılır.

3.6 Aradaki Elemanı Silmek ile Son Elemanı Silmek

Dikkat ederseniz son elemanı silmekle aradaki bir elemanı silmek arasında fark yoktur. İkisinde de aynı işlemler yapılır.

7 silinseydi `current->value` ilk bakışta 7'ye eşit olurdu ve döngüye hiç girilmeden çıkılırdı. `prev` 1000, `current` 1200 olduğundan `prev->next = current->next` ile 1000'in `next`'i 1500 olurdu. Ardından 1200 serbest bırakılırdı.

Tek farklı durum ilk elemanın silinmesidir. Çünkü o zaman `head` değişkeninin değerinin değişmesi gerekir. Önceki dersten hatırlayacağınız gibi, `head` ilk elemanın adresini tutan, düğüm tipinde bir gösterici değişkendi.

3.7 Algoritmanın Tamamı

Hocanın akışı düz metin olarak şöyledir:

```
oku: value
head == NULL ?
  T: yaz "Empty List"
  F: head->value == value ?
       T: tmp = head
          head = head->next
          free(tmp)
       F: prev = head
          current = head->next
          (current != NULL) && (current->value != value) ?
            T: prev = current
               current = current->next
               -> koşula geri dön
            F: current == NULL ?
                 T: yaz "not exist"
                 F: prev->next = current->next
                    free(current)
```

Hocanın başta verdiği 7, 9, 12 listesinde (1000, 1500, 1700) 15'i aramak, "eleman yok" dalını gösterir (ara adımlar eklendi):

Adım	`prev`	`current`	`current->value`	Sonuç
`head->value` = 7				$7 \neq 15$, ilk eleman değil
Başlangıç	1000	1500	9	$9 \neq 15$, ilerle
1. ilerleme	1500	1700	12	$12 \neq 15$, ilerle
2. ilerleme	1700	NULL	yok	`current` NULL, döngüden çık, "not exist"

Ek bilgi (derste anlatılmadı): Hoca akışı koda çevirmeyi öğrencilere bıraktı. Aşağıdaki fonksiyon akışın doğrudan çevirisidir, derste yazılmadı. `gcc -Wall` ile uyarısız derlendi. Boş liste, ilk eleman, aradaki eleman, son eleman, listede olmayan eleman ve tek elemanlı liste durumlarıyla çalıştırıldı, sonuçlar akışla aynı çıktı. C'de `delete` ayrılmış bir kelime değildir, ama C++ ile karışmaması için burada `deleteNode` adı kullanıldı.

```
void deleteNode(int value)
{
    NODE *tmp, *prev, *current;
    if (head == NULL)
    {
        printf("Empty List\n");
    }
    else if (head->value == value)
    {
        tmp = head;
        head = head->next;
        free(tmp);
    }
    else
    {
        prev = head;
        current = head->next;
        while (current != NULL && current->value != value)
        {
            prev = current;
            current = current->next;
        }
        if (current == NULL)
            printf("not exist\n");
        else
        {
            prev->next = current->next;
            free(current);
        }
    }
}
```

4. Dairesel Tek Yönlü Bağlı Liste

4.1 Neden Dairesel?

Şimdiye kadarki yapıda her elemanın tek bir bağlantısı vardı. Yani her eleman tek bir göstericiyle tek bir elemanı gösteriyordu. Buna tek yönlü bağlı liste (single linked list) denir.

Bazı durumlarda listenin dairesel (circular) olması gerekebilir, yani son elemanın listenin başını göstermesi istenebilir. Bunu kuyruklarda göreceğiz. Hoca, önümüzdeki hafta yığın ve kuy-

ruk anlatılırken bunun kuyrukta ne kadar avantajlı olduğunun görüleceğini söyledi.

Tek elemanlı dairesel listede eleman kendini gösterir. Birden çok eleman varsa son eleman yine en başı gösterir.

4.2 Başlangıç Noktası: `last`

Liste dairesel olsa da onu kullanabilmek için bir başlangıç noktasına ihtiyacımız vardır. Dairesel olmayan listede bu iş için `head` değişkeni vardı ve ilk elemanın adresini tutuyordu.

Hoca bunun bin bir türlü yolu olduğunu, nasıl isterseniz öyle yapabileceğinizi söyledi, ama şu çözümü önerdi: Son elemanın adresini saklarsanız ilk elemana da erişmiş olursunuz. Bu göstericiye `last` diyelim.

Soru: İlk eleman nerededir? **Cevap:** `last`'ın `next`'indedir. Son eleman ise `last`'ın kendisindedir.

- İlk eleman: `last->next`
- Son eleman: `last`

Böylece bu kez düğüm tipinde `head` değil, `last` değişkeni tutulur. Başlangıçta NULL'dur, yani liste boştur.

```
NODE *last = NULL;
```

> **Ek bilgi (derste anlatılmadı):** Bu seçimin kolaylığı, aşağıdaki iki ekleme fonksiyonunda görülür: ikisinde de döngü yoktur. Listede kaç eleman olursa olsun, hem başa hem sona, yalnız `last` ve `last->next` üzerinden birkaç atamayla ekleme yapılır. Dairesel olmayan, yalnız `head` tutulan listede ise sona eklemek için son elemana kadar yürümek gerekir.

4.3 Nereye Eklenir?

Eklemeyi üç türlü yapabilirsiniz: başa, sona ya da araya. Hangisinin seçileceği algoritmanın ihtiyacına bağlıdır.

- Liste bir kuyruğu gösteriyorsa yeni gelen kişi sona eklenir.
- Sıralı bir listede yeni eleman, sırayı bozmayacak şekilde araya yerleştirilir.

Bu, pazartesi gördüğümüz dairesel olmayan listeyle aynı mantıktır. Hoca `create` fonksiyonunu `insert`'ün içinde çağırmayı tercih etmediğini söyledi. Çünkü tasarladığınız algoritmada başka bir yerde oluşturulmuş bir düğümü de ekliyor olabilirsiniz. Bu yüzden fonksiyon, oluşturulmuş düğümü parametre olarak alır. Aynı yaklaşımı çift yönlü listede de sürdürdü ("dışarıdan alayım").

5. Dairesel Listede Başa Ekleme

5.1 Liste Boşsa

`last` NULL ise liste boştur. O zaman yeni düğüm hem ilk hem son elemandır. Hoca yeni düğümü 1000 numaralı adreste, henüz listeye bağlanmamış bir düğüm olarak çizdi. Düğüme değer olarak 5 yerleştirilir, sonra:

- `last = newNode`: `last` 1000 olur.
- `newNode->next = last`: 1000'in `next`'i 1000 olur. Düğüm dairesel olarak kendi kendini gösterir.

Soru: `last = newNode` satırını bir alt satıra indirsek daha iyi olmaz mıydı? Sonuçta `last`'ın değerini değiştirdik. **Cevap:** Olmazdı, çünkü o zaman `newNode->next` atanırken `last` hâlâ NULL olurdu ve `newNode`'un `next`'i NULL olurdu.

> **Ders sonrası not:** Hocanın bu cevabı derleyip çalıştırarak doğrulandı: Satırların sırası değiştirildiğinde düğümün `next`'i NULL kaldı.

`newNode->value = value` ataması liste boş da olsa dolu da olsa yapılacağı için hoca onu `if`'in dışına aldı. Düğümün `value`'suna elemanı bir kere koymak yeterlidir.

5.2 Liste Doluysa

Liste doluysa iki atama yapılır:

```
newNode->next = last->next;   /* yeni düğüm eski ilk elemanı gösterir */
last->next = newNode;         /* son eleman yeni ilk elemanı gösterir */
```

Hoca bunu önce 1000'de 5'in bulunduğu tek elemanlı listeye 1200'deki 7'yi ekleyerek gösterdi. `newNode->next = last->next` ile 7'nin `next`'i 1000 olur. `last->next = newNode` ile 1000'in `next`'i 1200 olur. `last` değişmez, hâlâ 1000'dir. Sonuçta 7 başa geçmiştir: 7, 5'i; 5 de 7'yi gösterir.

> **Ders sonrası not:** Hoca ilk `newNode->next = last->next` satırını anlatırken "bu yedi son eleman olacağı için ilk elemanı göstermeli" dedi; doğrusu başa eklemede 7 son eleman değil, yeni ilk elemandır ve eski ilk elemanı gösterdiği için `last->next`'i alır. İki elemanlı bu örnekte eski ilk eleman ile son eleman aynı düğüm olduğu için adres yine 1000 çıkar. Hoca karışıklığı kendisi de fark etti: Tahtanın üst kısmında sona eklemeyi, alt kısmında başa eklemeyi aynı sayılarla göstermişti.

Hoca aradan sonra, kafa karışıklığını gidermek için başka bir örnekle baştan anlattı.

5.3 Temiz Bir Örnek: 8, 4, 7

Başlangıçta `last` 1000'i gösteriyor. 1000'de 8 var ve kendini gösteriyor. Bu, boş listeye 8'in eklenmiş hâlidir.

4'ün eklenmesi. 1200 numaralı adres tutuldu ve oraya 4 yerleştirildi. Yeni düğüm başa eklenecektir, yani arkasına 8 gelecektir.

- `newNode->next = last->next`: `last` 1000, 1000'in `next`'i de 1000'dir (kendini gösteriyordu). 1200'ün `next`'i 1000 olur, yani 4, 8'i gösterir.
- `last->next = newNode`: 8 hâlâ kendini gösteriyordu, ama artık başı göstermesi gerekir. 1000'in `next`'i 1200 olur.
- `last` hâlâ 1000'dir, çünkü son eleman hâlâ 8'dir.

7'nin eklenmesi. 800 numaralı adres tutuldu ve oraya 7 yerleştirildi.

> **Hocanın vurgusu:** Başa eklemek demek, `last`'ın arkasına eklemek demektir. Bunu unutmayın.

İş bittiğinde olmasını istediğimiz şey şudur: 7'nin arkasında 4, arkasında 8, onun arkasında yine 7.

- `newNode->next = last->next`: `last->next` 1200'dür. 800'ün `next`'i 1200 olur, yani 7, 4'ü gösterir. Eskiden 4'ü 8 gösteriyordu, artık 7 gösterir.
- `last->next = newNode`: 1000'in `next`'i 800 olur. 8 artık 4'ü değil, 7'yi gösterir.
- `last` değişmez, 1000 kalır.

Durum	Liste (ilk elemandan başlayarak)	`last`	`last->next`
Boş listeye 8 (1000)	8 → 8	1000	1000
Başa 4 (1200)	4 → 8 → 4	1000	1200
Başa 7 (800)	7 → 4 → 8 → 7	1000	800

Tamamlanmış fonksiyon şöyledir:

```
NODE *last = NULL;

void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    if (last == NULL)            /* bağlı liste boş */
    {
        last = newNode;
        newNode->next = last;    /* tek düğüm kendini gösterir */
    }
```

```
    else
    {
        newNode->next = last->next;   /* yeni düğüm eski ilk elemanı gösterir */
        last->next = newNode;         /* son eleman yeni ilk elemanı gösterir */
    }
}
```

Tahtadaki kodda bir öğrenci sondan bir önceki satırdaki noktalı virgülün unutulduğunu fark etti. Yukarıdaki biçimde bu eksik giderilmiştir.

Soru: Hoca fonksiyonun sonuna önce `return last;` yazdı. Bir öğrenci sordu: `last` zaten global değil miydi, niye geri döndürdük? **Cevap:** Hoca haklı olduğunu söyledi ve dönüş tipini `void` yaptı. Fonksiyonu yazarken bir an geri döndüreceğini düşünmüştü, ama `last` global tanımlıydı ve parametre olarak da aktarılmamıştı. Değişkeni fonksiyondan fonksiyona aktarmak isterseniz o zaman döndürürsünüz.

> **Ek bilgi (derste anlatılmadı):** Dolu listede iki atamanın sırası önemlidir. Önce `last->next = newNode` yapılırsa eski ilk elemanın adresi kaybolur. Ardından gelen `newNode->next = last->next` yeni düğümü kendisine bağlar. Bu ters sıra derlenip denendi: Yeni düğüm gerçekten kendini gösterdi.

6. Dairesel Listede Sona Ekleme

Hoca, başa ve sona ekleme arasında çok ufak bir fark olduğunu göstermek için iki fonksiyonu yan yana yazdı. Liste boşsa baş ve son diye bir şey yoktur, bu yüzden o kısım aynen alınır.

Liste doluyken örnek şöyledir: 1000'de 5 var, 1500'de 9 var. 5, 1500'ü; 9 da 1000'i gösteriyor. `last` 1500'dür. Yeni eleman için 1800 numaralı adres tutuldu, orada 8 var ve şu anda NULL'u gösteriyor. 8 sona eklenecek.

Hoca önce "yavaş çekim" olarak ne olması gerektiğine baktı: 9 artık 5'i değil yeni geleni göstermelidir. Yeni gelen, 9'un gösterdiğini, yani 5'i göstermelidir. `last` da yeni gelen olmalıdır. Bunu üç aşamada yaparız:

- `newNode->next = last->next`: `last->next` 1000'dir. 1800'ün `next`'i 1000 olur, yani 8, 5'i gösterir.
- `last->next = newNode`: 1500'ün `next`'i 1800 olur, yani 9, 8'i gösterir. Hoca bunu öğrencilere sordu, cevap "new node" oldu.
- `last = newNode`: `last` 1500 değil 1800 olur.

Sonuç: 5 → 9 → 8 → 5.

İlk iki satır başa eklemedekiyle aynıdır. Başa eklerken son eleman değişmediği için `last`'a dokunulmuyordu. Sona eklediğimizde yeni gelen eleman son eleman olduğu için `last` değişir.

> **Hocanın vurgusu:** Başa ekleme ile sona ekleme arasındaki tek fark, sona eklerken `last = newNode` yapılmasıdır.

```
void insertAtLast(NODE *newNode, int value)
{
    newNode->value = value;
    if (last == NULL)
    {
        last = newNode;
        newNode->next = last;
    }
    else
    {
        newNode->next = last->next;
        last->next = newNode;
        last = newNode;               /* tek fark: yeni düğüm artık son eleman */
    }
}
```

Soru: Kelimenin sonunda “y” mi var, “e” mi? Yukarıda “single”, aşağıda “singly” yazıyor. **Cevap:** Hoca, dairesel listede “circular singly” dendiği için öyle yazdığını söyledi. Baktığında “circular single” da kullanıldığını gördü. Her iki kullanımın da mümkün olduğunu, İngilizcesi konusunda kesin konuşmak istemediğini ekledi.

7. Bellekteki Adreslerle İlgili Bir Soru

Soru: Son örnekte adresleri 1000, 1500, 1800 diye sıralı verdik. Önceki örnekte ise en son eklenen 7’ye en düşük adres olan 800’ü verdik. Bunun böyle olması gerekmiyor, değil mi?

Cevap: Gerekmiyor, tamamen tesadüf. Hoca sayıları gelişigüzel yazdığını, amacının tutulan yerlerin arka arkaya olması gerekmediğini göstermek olduğunu söyledi. Dinamik bellek ayırmada (dynamic memory allocation) neresi boşsa orası verilir. Gerçekte aralar bu kadar açık da olmaz. Genellikle boş yerler arka arkaya gelir. Örneğin 1000’de 12 bayt tutulduysa sonraki yer 1012, 1016 civarında olabilir.

Bellek gerçekte 1000 numaralı adres gibi küçük bir adres de vermez. Tutulan yeri görmek isterseniz göstericiyi `%p` ile yazdırabilirsiniz.

8. Dairesel Listeyi Yazdırmak

Hoca son örnek olarak listeyi yazdırmayı gösterdi. Geçici bir `p` göstericisi tutulur. `last` NULL ise yapacak bir şey yoktur. Değilse yazdırmaya başlanır.

Soru: İlk eleman hangi gözdedir, yazdırmaya nereden başlayacağız? **Cevap:** `last`’ın `next`’inden.

Tahtadaki örnekte `last` 1000 numaralı adresteydi. Hoca `last->next` için önce yanlışlıkla 1200 yazdığını fark etti ve düzeltti: İlk eleman 5000 numaralı adresteydi ve orada 9 vardı. Kodda da değişken adını önce `temp` yazıp sonra `p` yaptı.

Yazdırmaya ilk elemandan başlanır, tekrar ilk elemana gelince durulur. Son elemana geldiğimizde onun `next`’i yine `last->next`’tir. Hoca, elemanların sırayla 9, 7, 4 diye yazdırılacağını söyledi.

```
void printList()
{
    NODE *p;
    if (last == NULL)
        printf("List is empty\n");
    else
        for (p = last->next; p != last->next; p = p->next)
            printf("%d\n", p->value);
}
```

Düğümün bellekte nerede durduğunu görmek isterseniz `printf` satırını şöyle yazabilirsiniz. Böylece hem `p`’yi hem `p`’nin değerini görürsünüz:

```
printf("%p %d\n", p, p->value);
```

> **Ders sonrası not:** Hoca tahtaya yukarıdaki `for` döngüsünü yazdı ve elemanları sırayla yazdıracağını söyledi; doğrusu bu döngü hiçbir şey yazdırmaz. `p`, başlangıçta `last->next`’e eşitlenir ve koşul hemen `p != last->next` diye sorulduğu için döngüye hiç girilmez. Liste, ilk eleman yazdırıldıktan sonra kontrol eden bir `do-while` ile dolaşılmalıdır. Aşağıdaki kod derste yazılmadı. İki sürüm de `gcc -Wall` ile derlendi ve 9, 7, 4 listesinde çalıştırıldı: Tahtadaki hâli 0 satır yazdırdı, düzeltilmiş hâli 9, 7, 4 yazdırdı.
>
> ```
> void printList()
> {
>     NODE *p;
>     if (last == NULL)
>     {
>         printf("List is empty\n");
> ```

```
        return;
    }
    p = last->next;              /* ilk eleman */
    do
    {
        printf("%d\n", p->value);
        p = p->next;
    } while (p != last->next);   /* tekrar ilk elemana gelince dur */
}
```

9. Çift Yönlü Bağlı Liste

9.1 Neden Gerekir?

Tek yönlü listede her eleman yalnız bir başka elemanı gösterir. Hoca, istenirse öncekini de gösterebileceğini ama bunun pek anlamlı olmadığını söyledi. Mantıken gösterilen, kendisinden sonra gelen elemandır.

Oysa araya ekleme ya da silme yaparken kendisinden önceki elemanın yerini bilmek gerekir. Hoca iki kısa örnek verdi:

- Listede 3, 5, 4 sırasıyla duruyor olsun. 5 silinecekse 3'ün artık 4'ü göstermesi gerekir. Yani 5'ten önceki elemanın yeri bilinmelidir.
- Sıralı bir listede 2'nin ardından 5 gelsin. Sırayı bozmadan 4 eklemek istiyoruz. 4, 2'den büyük mü? Evet. 4, 5'ten büyük mü? Hayır. O zaman 4 araya eklenir. Bunun için 5'ten önceki elemanın yeri bilinmelidir ki o artık 5'i değil 4'ü göstersin.

Ayrıca bazı durumlarda elemanlar üzerinde ilerlerken tersten gitmek de istenebilir. Önceki elemanın yerini sürekli bilme ihtiyacı varsa, yapı elemanların hem sonrakini hem öncekini göstereceği şekilde kurulabilir. Buna çift yönlü bağlı liste (doubly linked list) denir.

9.2 Yapının Tanımı

Örnek: A 1000'de, B 1200'de, C 1080'de olsun.

Düğüm	Adres	Önceki	Sonraki
A	1000	NULL	1200
B	1200	1000	1080
C	1080	1200	NULL

A'yı `head` gösterir. A'dan önce kimse olmadığı için önceki göstericisi NULL'dur. C'den sonra kimse olmadığı için sonraki göstericisi NULL'dur.

Her düğümde iki gösterici vardır: önceki düğümün adresini tutan önceki göstericisi (previous pointer) ve sonraki düğümün adresini tutan sonraki göstericisi (next pointer). Yapıda zaten `struct node` tipinde bir `next` vardı. Buna `struct node` tipinde, kendisinden önceki elemanın adresini tutan bir `prev` eklenir:

```
struct node *next;
struct node *prev;
```

> **Hocanın vurgusu:** Unutmamanız gereken şey, ilk elemanın `prev`'inin her zaman NULL olmasıdır, çünkü ilk elemandan önceki eleman yoktur. Son elemanın `next`'i de NULL'dur.

İstenirse çift yönlü liste de dairesel yapılabilir. Hoca bunu tahtada gösterecek gibi oldu, ama kafa karışmasın diye geri aldı ve bu derste anlatmadı.

9.3 Avantajı ve Dezavantajı

	Açıklama
Avantaj	Sürekli geçici bir `prev` değişkeninde önceki elemanın yerini tutmak gerekmez. Herhangi bir anda bir elemanın önceki elemanının adresi görülebilir.
Dezavantaj	Tek yönlü listede silerken ve eklerken önceki adresi tutan tek bir geçici gösterici yetiyordu. Burada her düğüm için ayrı bir gösterici değişkeni tutulur. Bu yer kaybıdır.

> **Hocanın vurgusu:** Çift yönlü liste yapacaksanız bundan çok emin olmalısınız. Algoritmanız bütün elemanların önceki elemanlarının yerini sürekli kullanmaya ihtiyaç duyuyorsa, yani o anki elemanın öncesini geçici olarak bulmak yetmiyorsa çift yönlü yaparsınız. Yoksa problemi tek bir geçici değişkenle çözebilirsiniz.

10. Çift Yönlü Listede Başa Ekleme

Hoca yine düğümü dışarıdan alan bir fonksiyon yazdı. Başa ekleme yapıyoruz:

- **Liste boşsa:** `head` yeni düğüm olur. Bu ilk düğümün `prev`'i de NULL olmalıdır.
- **Liste boş değilse:** `head`'in `prev`'i yeni düğüm olur. Yeni düğümün `next`'i `head` olur. `head` de yeni düğüm olur.

```
void insertAtFront(NODE *newNode, int value)
{
    newNode->value = value;
    if (head == NULL)
    {
        head = newNode;
        /* head->prev = NULL;  create içinde yapıldığı için gerek yok */
    }
    else
    {
        head->prev = newNode;
        newNode->next = head;
        head = newNode;
    }
}
```

Hocanın örneği: `head` 1000'i gösteriyor. 1000'de 5 var, 1200'de 7 var. 1000'in öncesi NULL, sonrası 1200. 1200'ün öncesi 1000, sonrası NULL. Yeni eleman 13, 1500 numaralı adreste duruyor. `create` edilirken iki göstericisi de NULL yapılmış.

`head` NULL mı? Hayır, 1000. O zaman:

- `head->prev = newNode`: 1000'in, yani 5'in `prev`'i 1500 olur.
- `newNode->next = head`: 13'ün `next`'i 1000 olur.
- `head = newNode`: `head` 1500 olur.

Sonuçta `head` 13'ü, 13 de 5'i gösterir. 13'ün öncesi NULL'dur.

Düğüm	Adres	`prev`	`next`
13	1500	NULL	1000
5	1000	1500	1200
7	1200	1000	NULL

> **Ders sonrası not:** Bu fonksiyon `gcc -Wall` ile derlenip çalıştırıldı ve tablodaki bağlantılar doğrulandı: Liste ileri yönde 13, 5, 7; geri yönde 7, 5, 13 olarak dolaşıldı.

Soru: Boş liste durumunda `head->prev = NULL` yaptık. Yalnız `prev`'i NULL yapıp `next`'i yapmamamızın özel bir nedeni var mı? **Cevap:** `next` zaten NULL'dur, çünkü `create` içinde öyle

ayarlanmıştır. Hoca öğrencinin haklı olduğunu, aslında `prev` satırına da gerek olmadığını söyledi. Bu satırı silip önceki dersteki `create` fonksiyonuna tek bir ekleme yaptı:

```
NODE *create()
{
    /* ... yer ayır (allocate) ... */
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}
```

`create` içinde `prev` de NULL yapılırsa ekleme fonksiyonlarında bunu ayrıca yazmaya gerek kalmaz. Hoca o satırı görüntü olsun diye yazdığını söyledi ve "yapmayın" dedi.

Sona ve araya ekleme öğrencilere bırakıldı.

Dersin Toparlaması

Bu derste bağlı liste üzerinde silme tamamlandı. Silme, önce durumları belirlemenin neden işe yaradığını gösteren bir örnek oldu: Liste boş olabilir, silinecek eleman ilk eleman olabilir, arada ya da sonda olabilir, listede hiç olmayabilir. Aradaki ve sondaki eleman için işlem aynıdır: `prev->next = current->next` ve `free(current)`. Yalnız ilk eleman silinirken `head` değişir. Döngüde de döngüden çıkışta da önce `current`'ın NULL olup olmadığına bakılır.

Ardından iki yeni yapı görüldü. Dairesel tek yönlü listede son eleman ilk elemanı gösterir ve `last` göstericisi tutulur: İlk eleman `last->next`, son eleman `last`'tır. Başa ve sona ekleme yalnız `last = newNode` satırıyla ayrılır. Çift yönlü listede her düğümde `prev` ve `next` vardır. Bu, önceki elemana her an erişim sağlar, ama her düğümde bir gösterici kadar yer harcar. Bu yüzden ancak gerçekten ihtiyaç varsa seçilir. Önümüzdeki hafta anlatılacak yığın ve kuyrukta bu liste mantığı kullanılacak, dairesel yapının kuyruktaki avantajı da orada görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.